PHYB57 科学计算导论

作业二：中文完整解答

递归与复杂度 · 双精度浮点数 · 线性最小二乘回归

姓名：_____ 学号：_____

本讲义依据原题及所附 100 组实验数据，按题号给出推导、关键 Python 代码、计算结果及图表。 分析结果与此前已运行的 Jupyter Notebook 一致。

主要答案

题目	结果
第一题：直接递归	时间复杂度 $\Theta(n^{\log_2\varphi}) \approx \Theta(n^{0.69424})$
第一题：滚动迭代	时间复杂度 $\Theta(\log n)$；保留常数个状态变量
第二题：存储的 0.1	十六进制位型 `0x3FB9999999999999A`
小数点后前 20 位	`0.10000000000000000555`
前一个可表示数	十六进制位型 `0x3FB9999999999999`
第三题：拟合直线	$\widehat{y} = 1.451647164716x + 0.559818181818$
第三题：拟合优度	$R^2 = 0.999992972125$

其中 $\varphi = (1+\sqrt{5}\,)/2$。第一题的迭代复杂度针对本文明确给出的滚动算法；“迭代”一词本身不能唯一决定复杂度。

阅读顺序

1. 第一题先区分函数值 $t(n)$ 与运行时间 $T(n)$，再推导递归树和迭代次数。
2. 第二题依次求二进制展开、指数栏、尾数舍入、精确存储值和相邻浮点数。
3. 第三题先检查数据，再推导最小二乘公式，最后结合散点图与残差图解释结果。

材料与运行环境

原题：`PHYB57_problem_set_2.pdf`，第 1–2 页。 数据：`Dataset.xlsx`，`Sheet1!A1:B101`，表头为 `x`、`y`。

复现时将数据文件与 Notebook 放在同一文件夹；需要 `numpy`、`pandas`、`matplotlib` 和 `openpyxl`。本 PDF 是便于阅读和打印的中文解答，代码可在原 Notebook 中编辑运行。

> AI 使用说明：本解答使用 OpenAI Codex 辅助编写推导与 Python 代码、执行计算，并生成图表和文档。原题要求说明 AI 工具的使用方式，此处据实保留。

第一题：递归的时间复杂度

题目给出的函数为

$$t(n) = t(n/2) + t(n/4), \qquad t(n) = 1 \quad (n < 1).$$

以下假设输入 n 是非负整数，并采用入门算法分析常用的单位成本模型：一次基本算术操作计为常数时间。需要注意，停止条件是 $n < 1$，所以 $t(1) = t(1/2) + t(1/4) = 2$。

1. 先区分函数值和运行时间

$t(n)$ 表示函数返回的数值；$T(n)$ 表示计算该数值需要的时间。 直接递归会计算两个子问题，另加判断和加法等常数工作量，因此

$$T(n) = T(n/2) + T(n/4) + \Theta(1).$$

最长分支只有对数深度，但总时间取决于**整棵调用树**，不能把深度直接当作运行时间。

2. 令输入为 2 的幂，化成 Fibonacci 递推

令 $n = 2^k$，并设 $u_k = t(2^k)$。此时

$$u_k = u_{k-1} + u_{k-2}, \qquad u_{-2} = u_{-1} = 1.$$

于是 $u_0 = 2, u_1 = 3, u_2 = 5, u_3 = 8$。若 $F_0 = 0, F_1 = 1$，则

$$u_k = F_{k+3}.$$

Fibonacci 数列的特征方程为 $\lambda^2 = \lambda + 1$，正根是 $\varphi = (1 + \sqrt{5})/2$，所以 $u_k = \Theta(\varphi^k)$。代入 $k = \log_2 n$：

$$\varphi^k = \varphi^{\log_2 n} = n^{\log_2 \varphi}.$$

3. 从返回值联系到调用次数

每个叶节点返回 1，每个内部节点把两个子节点的结果相加。因此叶节点数恰好是 $t(n)$。 每个内部节点都有两个子节点，所以当叶节点数为 L 时，总节点数为 $2L - 1$。

$$T_{\text{递归}}(n) = \Theta(n^{\log_2 \varphi}) = \Theta(n^{0.6942419136\ldots}).$$

对任意整数 $n \geq 1$，还可以写成 $t(n) = F_{\lfloor \log_2 n \rfloor + 3}$， 所以不只在 2 的幂处成立。递归栈的最大深度为 $\Theta(\log n)$。

第一题：迭代算法与代码核验

1. 每个“减半层级”只计算一次

利用前一页的 Fibonacci 形式，只保留前两项。 初始化 $u_{-2} = u_{-1} = 1$，每跨过一个减半层级就更新一次。

```
def t_recursive(n):
    # 对非负整数，整除与精确除法具有相同的停止层级。
    if n < 1:
        return 1
    return t_recursive(n // 2) + t_recursive(n // 4)


def t_iterative(n):
    previous_two, previous_one = 1, 1
    while n >= 1:
        previous_two, previous_one = (
            previous_one, previous_two + previous_one
        )
        n //= 2
    return previous_one


for n in range(257):
    assert t_recursive(n) == t_iterative(n)
```

这里的整除不会改变非负整数输入的答案：任何分支只会不断除以 2 的幂， 而“精确值小于 1”与“整除结果为 0”在同一层发生。

2. 迭代时间与空间

当 $n \geq 1$ 时，循环执行 $\lfloor \log_2 n \rfloor + 1$ 次。因此

$T_{\text{迭代}}(n) = \Theta(\log n).$

算法只保留两个递推状态，所以在单位成本模型中，辅助空间为常数个存储值。 如果细算 Python 大整数的位数与加法成本，就不能再把任意大的整数当作固定大小对象。

输入 n	函数值 $t(n)$	直接递归调用数	迭代更新数
1	2	3	1
16	13	25	5
256	89	177	9
4096	610	1219	13
1048576	28657	57313	21

表中的递归调用数满足 $2t(n) - 1$。代码已核验 $n = 0, 1, \ldots, 256$ 的两种实现结果一致。

第一题：图示与易错点

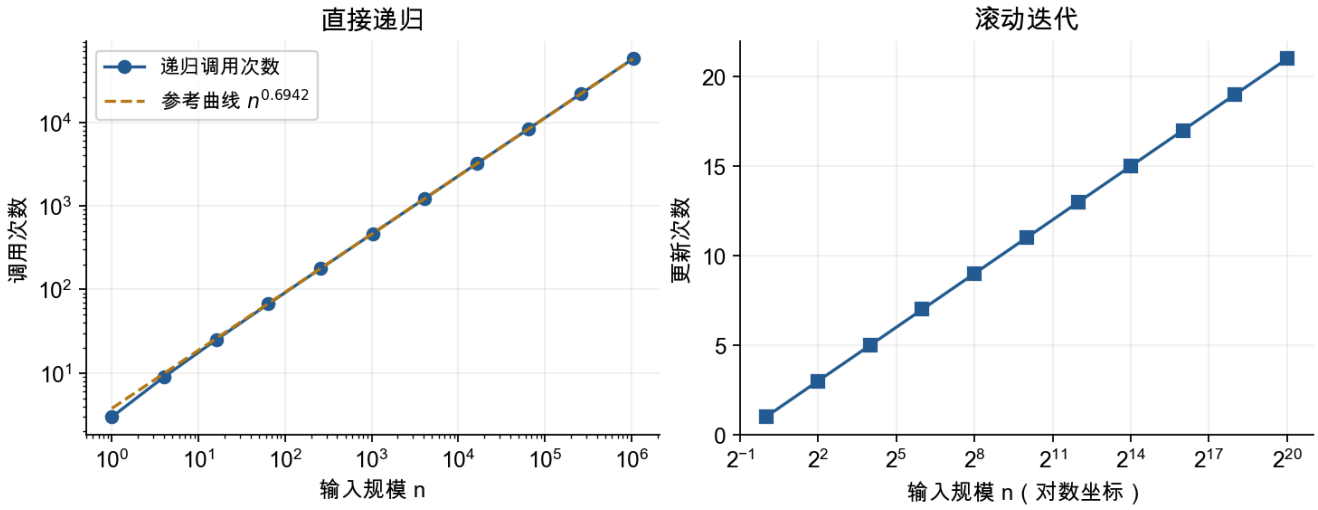


图 1： 左图直接统计递归调用次数，并给出相同增长阶的参考曲线； 右图统计滚动算法的更新次数。这些是确定性的操作计数，不是依赖电脑性能的计时结果。

为什么“迭代”不一定都是对数时间?

实现方式	做了什么	时间复杂度
直接递归	重复展开两个子问题	$\Theta(n^{\log_2 \varphi})$
本文的滚动迭代	每个减半层级更新一次	$\Theta(\log n)$
按整数逐项填表	从 0 一直计算到 n	$\Theta(n)$
用显式栈模拟递归	仍然遍历原来的调用树	$\Theta(n^{\log_2 \varphi})$

因此答题时应同时给出迭代写法或明确算法逻辑，再报告复杂度。 本文使用递推式的特殊结构，只计算必要的层级。

三个关键提醒

1. 两个子问题的规模分别是 $n/2$ 与 $n/4$，不能直接当成两个 $n/2$ 的子问题。
2. 返回值 $t(n)$ 与运行时间 $T(n)$ 是不同概念；本题利用递归树才建立了二者的增长关系。
3. 原题基例为 $n < 1$。若误写为 $n \le 1$，会改变数值答案，虽然主要增长阶仍相同。

第二题：0.1 的双精度位表示

1. 十进制小数转二进制

对小数部分反复乘以 2，取整数部分作为下一位：

步骤	当前余数乘以 2	取出的位	新余数
1	0.2	0	0.2
2	0.4	0	0.4
3	0.8	0	0.8
4	1.6	1	0.6
5	1.2	1	0.2

从第 5 步开始，余数再次出现 0.2，所以后续会循环。于是

$$0.1_{10} = 0.00011001100110011\ldots_2 = 1.10011001100110011\ldots_2 \times 2^{-4}.$$

2. 分别填写符号、指数和 fraction

binary64 包含 1 位符号、11 位指数和 52 位存储 fraction。 规格化数的首位 1 隐含不存，所以有效精度为 53 位。

符号位 $s = 0$。实际指数 $e = -4$，加上偏置 1023 后：

$$E = e + 1023 = 1019 = (01111111011)_2.$$

3. 决定最后一位如何舍入

该指数区间的间距是 2^{-56}。把精确实数 $1/10$ 放到这个网格上：

$$\frac{1}{10}\,2^{56} = 7205759403792793.6.$$

按“舍入到最近值，恰好居中时取偶数”的规则，得到 $M = 7205759403792794$。这里并非恰好居中；向上舍入更近。

$$f = M - 2^{52} = 2702159776422298.$$

最终 64 位表示为：

```
符号：0
指数：01111111011
尾数：1001100110011001100110011001100110011001100110011001100110011010
```

十六进制位型： `0x3FB9999999999999A`。 注意：此处“尾数”指存储的 52 位 fraction，不包括隐含的首位 1。

第二题：精确存储值与前一个浮点数

1. 从位型恢复精确数值

规格化 binary64 数的数值为

$$(-1)^s\left(1+\frac{f}{2^{52}}\right)2^{E-1023}.$$

代入本题参数：

$$x_{\text{存储}}=\frac{7205759403792794}{2^{56}}=\frac{3602879701896397}{36028797018963968}.$$

其精确十进制值为：

```
0.1000000000000000055511151231257827021181583404541015625
```

所以小数点后的前 20 位是 `10000000000000000555`，截取到该位为 **`0.10000000000000000555`**。存储值比精确的十进制 0.1 略大，误差约为 $5.5511151231\times10^{-18}$。

2. 相邻浮点数的间距

在 $[2^{-4},2^{-3})$ 内，相邻 binary64 值的间距为

$$\text{ULP}=2^{-4-52}=2^{-56}.$$

```
2^-56 = 0.000000000000000013877787807814456755295395851135253906 25
```

因此前一个可表示数为

$$x_{-}=x_{\text{存储}}-2^{-56}=\frac{7205759403792793}{72057594037927936}.$$

其精确值及位表示为：

```
0.09999999999999999167332731531132594682276248931884765625

符号：0
指数：01111111011
尾数：0011001100110011001100110011001100110011001100110011001
十六进制：0x3FB9999999999999
```

由于 $x_{-}<1/10<x_{\text{存储}}$，这个数同时也是严格小于精确实数 0.1 的最大 binary64 数。

> 不要直接减去机器 epsilon 2^{-52}：它描述的是 1 附近的间距。本题指数为 −4，应使用 2^{-56}。Python 默认显示的 `0.1` 也不代表精确存储值只有一位小数。

第二题：用 Python 验证

用 `struct` 读取真实位型，用 `Fraction` 和 `Decimal` 恢复精确值； `math.nextafter` 沿浮点数网格移动到前一个可表示数。

```
import struct
import math
from fractions import Fraction
from decimal import Decimal

stored = 0.1
word = struct.unpack('>Q', struct.pack('>d', stored))[0]
bits = f'{word:064b}'

print(bits[:1], bits[1:12], bits[12:], sep=' | ')
print(f'0x{word:016X}')
print(Fraction.from_float(stored))
print(Decimal.from_float(stored))

text = format(Decimal.from_float(stored), 'f')
print(text.split('.')[1][:20])

previous = math.nextafter(stored, -math.inf)
previous_word = struct.unpack(
    '>Q', struct.pack('>d', previous)
)[0]
print(f'0x{previous_word:016X}')
print(Decimal.from_float(previous))

gap = Fraction.from_float(stored) - Fraction.from_float(previous)
assert gap == Fraction(1, 2**56)
assert previous_word == word - 1
assert math.nextafter(previous, math.inf) == stored
```

主要输出

```
0 | 01111111011 | 1001100110011001100110011001100110011001100110011001100110011010
0x3FB999999999999A
3602879701896397/36028797018963968
0.1000000000000000055511151231257827021181583404541015625
1000000000000000555
0x3FB9999999999999
0.09999999999999999916733273153113259468227624893188476562 5
```

这里要用 `Decimal.from_float(stored)` 查看已有浮点数的精确值。 `Decimal('0.1')` 表示精确的十进制 0.1，是不同的对象。

第三题：数据检查与最小二乘推导

1. 数据范围与建模选择

数据共有 100 组 (x_i, y_i)，且 $x_i = 1, 2, \ldots, 100$。 源表的数值以文本保存，读取后显式转为数值。没有缺失单元格或重复数据行，全部观测都保留。 原题未提供单位与各点的测量不确定度，因此使用等权普通最小二乘。

取线性模型

$$\widehat{y}_i = mx_i + b.$$

题面没有要求直线通过原点，所以保留截距 b。 最小化所有竖直残差的平方和：

$$S(m, b) = \sum_{i=1}^{N} [y_i - (mx_i + b)]^2.$$

2. 对两个参数分别求偏导

$$\frac{\partial S}{\partial m} = -2 \sum_i x_i [y_i - (mx_i + b)] = 0,$$

$$\frac{\partial S}{\partial b} = -2 \sum_i [y_i - (mx_i + b)] = 0.$$

整理为正规方程：

$$m \sum_i x_i^2 + b \sum_i x_i = \sum_i x_i y_i, \qquad m \sum_i x_i + Nb = \sum_i y_i.$$

解得

$$m = \frac{N \sum_i x_i y_i - (\sum_i x_i)(\sum_i y_i)}{N \sum_i x_i^2 - (\sum_i x_i)^2}, \qquad b = \bar{y} - m\bar{x}.$$

3. 代入所给数据

$$N = 100, \quad \sum x_i = 5050, \quad \sum y_i = 7386.8,$$

$$\sum x_i^2 = 338350, \quad \sum x_i y_i = 493991.9.$$

因此

$$m = \frac{100(493991.9) - 5050(7386.8)}{100(338350) - 5050^2} = \frac{12095850}{8332500} = 1.451647164716\ldots,$$

$$b = 73.868 - 50.5m = 0.559818181818\ldots.$$

$$\widehat{y} = 1.451647164716x + 0.559818181818.$$

第三题：Python 拟合与数值结果

设计矩阵 A 的第一列为 x，第二列全为 1。第二列对应截距。 使用 `np.linalg.lstsq` 直接解矩形最小二乘问题，避免显式求逆。

```
import numpy as np
import pandas as pd

raw = pd.read_excel('Dataset.xlsx', sheet_name='Sheet1', dtype=str)
data = raw.apply(pd.to_numeric, errors='raise')
x = data['x'].to_numpy(dtype=float)
y = data['y'].to_numpy(dtype=float)
assert len(data) == 100
assert np.isfinite(data.to_numpy(dtype=float)).all()

A = np.column_stack((x, np.ones_like(x)))
(m, b), _, rank, _ = np.linalg.lstsq(A, y, rcond=None)
assert rank == 2

y_fit = m*x + b
residuals = y - y_fit
N = len(x)
SSE = np.dot(residuals, residuals)
SST = np.sum((y - y.mean())**2)
RMSE = np.sqrt(SSE/N)
R_squared = 1 - SSE/SST
residual_standard_error = np.sqrt(SSE/(N - 2))

assert np.allclose(A.T @ residuals, np.zeros(2), atol=1e-8)
print(m, b, SSE, RMSE, R_squared)
```

量	数值
斜率 m	1.451647164716
截距 b	0.559818181818
残差平方和 SSE	1.234026642664
均方根误差 RMSE	0.111086751805
残差标准误差	0.112214565002
决定系数 R^2	0.999992972125

其中 $\mathrm{RMSE} = \sqrt{\mathrm{SSE}/N}$ ； 残差标准误差使用 $N-2$ 作分母， 因为拟合了两个参数。二者不要混淆。

计算已与前一页的求和公式交叉核验。另用精确分数重新计算，得到 $m = 80639/55550$、$b = 3079/5500$，与数值求解一致。

第三题：拟合图、残差图与解释

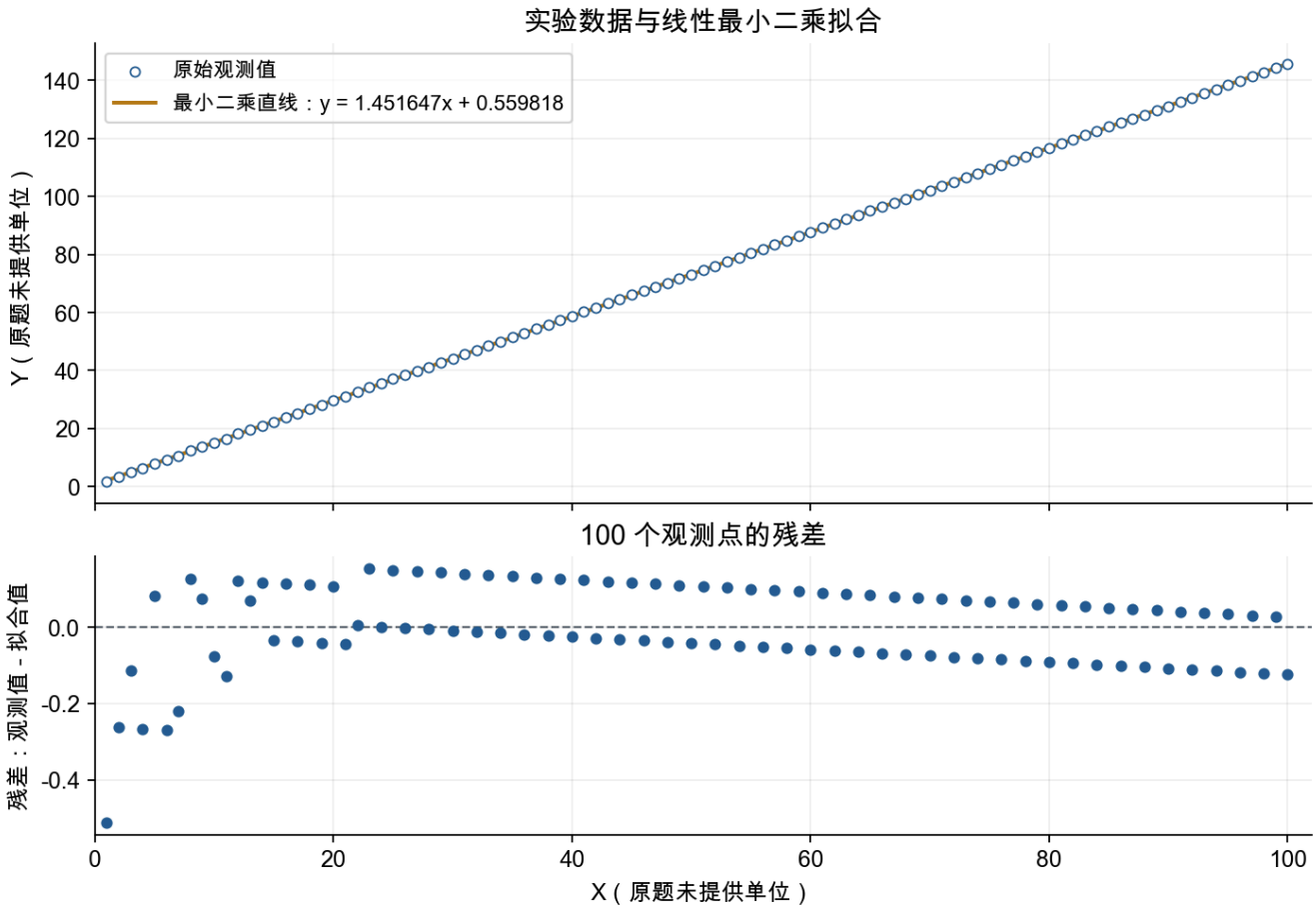


图 2： 上图包含全部 100 个观测点及拟合直线；下图显示相同横坐标上的带符号残差。 虚线 $r = 0$ 表示观测值与拟合值相等。原题未给单位，坐标轴据实注明。

如何理解结果?

斜率约为 1.451647，表示在这个模型中，X 每增加 1，预测的 Y 约增加 1.451647。 直线很好地描述了 $1 \le X \le 100$ 内的整体变化，R^2 约为 0.999993。

但残差在大部分区间呈现交替规律，且 $X = 1$ 处的负残差较大，约为 −0.5115。 因此，高 R^2 并不保证误差独立、随机，也不能证明直线就是精确物理定律。 现有材料无法确定这些残差结构的成因，不应凭图自行补充实验机制。

作答时应保留的结论

给出模型、最小二乘推导和参数数值；同时展示观测点与拟合直线。 残差图用于补充检查：整体拟合很好，但仍有系统结构值得进一步研究。 本解答使用全部数据，没有删除 $X = 1$ 的点，也没有对观测区间以外作外推。